בית הספר להנדסה ומדעי המחשב ע"ש רחל וסלים בנין

מבוא למדעי המחשב 67102 – סמסטר ב' 2024

תרגיל 6 – עיבוד תמונה ועבודה עם רשימות רב-מימדיות

להגשה בתאריך **25/06/2024** בשעה 22:00

הקדמה

בתרגיל זה נכתוב תכנית לעריכת תמונות. במהלך כתיבת התכנית נתרגל שימוש בלולאות ורשימות רב מימדיות וניחשף לכלים בסיסיים בעיבוד תמונה. התרגיל מורכב בצורה מובנית ממספר משימות, שבסופו של דבר יתחברו ביחד וירכיבו את המוצר הסופי.
שימו לב - **אין צורך בידע קודם בעיבוד תמונה**, את כל הידע הנדרש ריכזנו עבורכם בסרטון וידאו הנגיש באתר הקורס, **צפו בו ורק לאחר מכאן המשיכו בקריאה.**

עקבו אחר ההוראות והשלבים של התרגיל, והקפידו לכתוב את הקוד שלכם במדויק על פי הנחיות התרגיל. כמו כן, מומלץ בחום לקרוא את כלל התרגיל לפני תחילת הפתרון. אנו ממליצים להתחיל לעבוד על התרגיל בשלב מוקדם שכן התרגיל ארוך מקודמיו.
לרשותכם קובץ בשם **image_editor.py** עם חתימות הפונקציות שעליכם לממש בתרגיל. **אין לשנות חתימות אלו**.
הוספנו type hints לחתימות הפונקציות כדי להבהיר מה הם הטיפוסים של הפרמטרים וערכי ההחזרה של הפונקציות.
אתם רשאים לממש פונקציות עזר נוספות משלכם, ולהשתמש בהן בקוד שלכם. לאורך כל התרגיל ניתן להניח קלט תקין, אלא אם צוין אחרת.

הקובץ ex6_helper.py

בקובץ העזר מימשנו עבורכם מספר פונקציות שיעזרו לכם בטעינת התמונה ובצפייה בתמונות השונות הנוצרות במהלך התוכנית שתכתבו:

`load_image(image_path)`

מקבלת ניתוב לתמונה צבעונית ומחזירה ייצוג של התמונה כרשימה תלת-מימדית כפי שתיארנו בתרגול.

`save_image(image, path)`

מקבלת תמונה וניתוב ושומרת את התמונה בניתוב הנתון.

`show_image(image)`

מקבלת תמונה ומציגה אותה.

שימו לב: אין לעשות שום שינוי בקובץ זה ואין להגישו.

הספרייה PIL

הקובץ ex6_helper.py – הנתון לכם בתרגיל זה עושה שימוש בספריה PIL של פייתון, ולכן צריך אותה על מנת להריץ את התרגיל. **במעבדת המחשבים של האוניברסיטה (האקווריום) כבר מותקנת ספריה זו, ואין צורך להתקין שום דבר.**
לעבודה על המחשב האישי, בדקו אם החבילה מותקנת (היא כלולה ב-WinPython) ואם לא התקינו אותה באמצעות הפקודה:

```
python3 -m pip install Pillow
```

במידה וההתקנה נכשלה, יתכן שעליכם לעדכן את גרסת ה pip. הריצו את הפקודות הבאות:

```
python3 -m pip install --upgrade pip
python3 -m pip install --upgrade Pillow
```

עבודה עם תמונות צבעוניות

הפרדה לערוצי צבע

תמונה צבעונית מורכבת מרשימה תלת-מימדית כאשר המימד האחרון מתאר את כל הצבעים של פיקסל מסוים. במקרה הנפוץ עובדים עם ייצוג RGB – כלומר ישנם שלושה ערכים שמייצגים את רמת הבהירות של אדום, ירוק וכחול.

בתרגיל, כשנעבוד עם תמונות צבעוניות, נעבוד על כל **ערוץ צבע** בנפרד, ולכן ניצור מטריצה אחת שמכילה את כל הגוונים האדומים, אחת שמכילה את כל הירוקים ואחת שמכילה את כל הכחולים, וכך נעבוד על רשימות דו-מימדיות.

1. נפריד תמונות צבעוניות לערוצים נפרדים, ונחבר אותם חזרה לתמונה צבעונית.

- ממשו את הפונקציה:

`separate_channels(image)`

שמקבלת תמונה (רשימה תלת-מימדית) שמימדיה rows × columns × channels ומחזירה רשימה תלת-מימדית, שמימדיה channels × rows × columns, כלומר רשימה של תמונות דו-מימדיות שכל אחת מייצגת ערוץ צבע בודד.

ניתן להניח שהפונקציה מקבלת תמונה צבעונית חוקית. אין לשנות את תמונת המקור.

לדוגמא:

```
separate_channels([[[1, 2]]]) → [[[1]], [[2]]]

image = [[[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]]]
image_lst = [[[1, 1, 1], [1, 1, 1], [1, 1, 1], [1, 1, 1]],
             [[2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]],
             [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]]
separate_channels(image) → image_lst
```

- ממשו את הפונקציה:

`combine_channels(channels)`

שעושה את ההפך מהפונקציה הקודמת, כלומר מקבלת רשימה באורך channels של תמונות דו-מימדיות המורכבות מערוצי צבע בודדים, ומאחדת אותם לכדי תמונה צבעונית אחת שמימדיה rows × columns × channels.

ניתן להניח שהפונקציה מקבלת קלט תקין, של רשימה לא ריקה של תמונות דו- מימדיות (כלומר קיים לפחות ערוץ אחד). אין לשנות את רשימת המקור.

לדוגמא:

```
combine_channels([[[1]], [[2]]]) → [[[1, 2]]]
```

```
image_lst = [[[1, 1, 1], [1, 1, 1], [1, 1, 1], [1, 1, 1]],
             [[2, 2, 2], [2, 2, 2], [2, 2, 2], [2, 2, 2]],
             [[3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]]]
image = [[[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]],
         [[1, 2, 3], [1, 2, 3], [1, 2, 3]]]
separate_channels(image_lst) → image
```

<u>שימו לב</u>: במקרה שתיארנו (RGB) יש שלושה ערוצי צבע, אבל לא תמיד זהו המצב. יש עוד מרחבי צבעים ועוד ייצוגים של תמונות בהם עשוי להיות מספר שונה של ערוצי צבע. כתבו את הפונקציות באופן גנרי.

מעבר לגווני שחור לבן

תמונה בגווני שחור לבן היא תמונה המורכבת מערוץ יחיד, כמו בדוגמא הבאה: `[[[1], [2]], [[3], [4]]]`.

נהוג לוותר על המימד השלישי (של ה channels) בייצוג של תמונות עם ערוץ יחיד ולהשתמש ברשימה דו-מימדית (במקום תלת-מימדית). למשל את התמונה בדוגמא נייצג ע"י הרשימה הדו-מימדית הבאה: `[[1, 2], [3, 4]]` במקום.

מעתה ואילך בתרגיל, נייצג תמונות המורכבות מערוץ יחיד, ובפרט תמונות בגווני שחור לבן <u>ברשימות דו-מימדיות</u>.

2. ממשו את הפונקציה:

`RGB2grayscale(colored_image)`

הפונקציה מקבלת תמונה צבעונית (**רשימה תלת מימדית** כפי שהוסבר לעיל) ומחזירה תמונה בגווני שחור לבן (**רשימה דו-מימדית**).

ניתן להניח שהפונקציה מקבלת תמונה צבעונית חוקית בפורמט **RGB** (כלומר כל פיקסל מורכב מ-3 ערכים).

<u>אין לשנות את תמונת המקור.</u>

הפיכת תמונה צבעונית לתמונה בגווני שחור לבן נעשית על ידי מיצוע מסויים של ערכי הפיקסלים הצבעוניים לכדי ערך אחד. הנוסחא בה נשתמש היא:

$$RED \cdot 0.299 + GREEN \cdot 0.587 + BLUE \cdot 0.114$$

כאשר <u>יש לעגל את התוצאה לשלם הקרוב ביותר</u>.

לדוגמא:

```
RGB2grayscale([[[100, 180, 240]]]) → [[163]]
RGB2grayscale([[[200, 0, 14], [15, 6, 50]]]) → [[61, 14]]
```

טשטוש

ראינו בתרגול כיצד מטשטשים תמונה באמצעות שימוש בקרנל. כעת נממש טכניקה זו.

3. ממשו את הפונקציה:

`blur_kernel(size)`

המחזירה קרנל החלקה בגודל size × size ברשימה של רשימות.

קרנל ההחלקה בו נשתמש בתרגיל לא יהיה הקרנל הגאוסיאני שראינו בסרטון, אלא קרנל הממצע את כל השכנים בצורה שווה, כלומר כל תא בקרנל מכיל את הערך $\frac{1}{\text{size}^2}$.

ניתן להניח כי size הינו מספר שלם, חיובי **ואי זוגי.**

לדוגמא:

```
blur_kernel(3) → [[1/9, 1/9, 1/9], [1/9, 1/9, 1/9], [1/9, 1/9, 1/9]]
```

4. ממשו את הפונקציה:

apply_kernel(image , kernel)

הפונקציה מקבלת תמונה בעלת ערוץ צבע יחיד (קרי רשימה דו-מימדית) וקרנל (גם הוא רשימה דו-מימדית), ומחזירה תמונה בגודל זהה לזה של התמונה המקורית, כאשר כל פיקסל בתמונה החדשה מחושב באמצעות הפעלת הקרנל עליו.

כלומר:

מזהים את הפיקסל image[row][column] עם הכניסה המרכזית במטריצה kernel, וסוכמים את ערכי שכניו (כולל הפיקסל עצמו) כפול הכניסה המתאימה להם ב-kernel (ראו סרטון).

ניתן להניח שהפונקציה מקבלת תמונה חוקית בעלת ערוץ צבע יחיד (רשימה דו-מימדית), ושהקרנל בגודל $size \times size$ כאשר $size$ מספר טבעי **אי-זוגי**.

אין לשנות את תמונת המקור.

לדוגמא:

```
apply_kernel([[0, 128, 255]], blur_kernel(3))  →  [[14, 128, 241]]
```

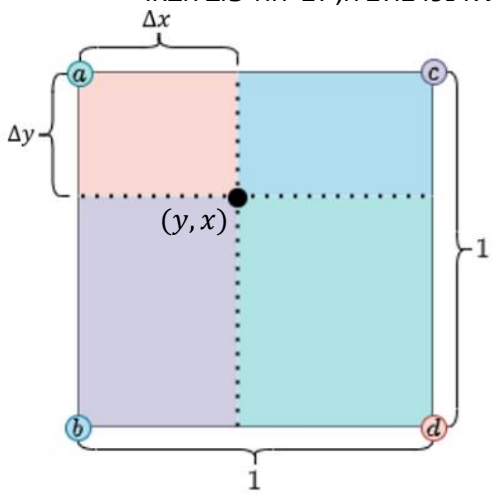
```
apply_kernel([[10, 20, 30, 40, 50],                    [[12, 20, 26, 34, 44],
              [8, 16, 24, 32, 40],                      [11, 17, 22, 27, 34],
              [6, 12, 18, 24, 30],                  →   [10, 16, 20, 24, 29],
              [4, 8, 12, 16, 20]], blur_kernel(5))      [7, 11, 16, 18, 21]]
```

שימו לב:

- **במידה והסכום אינו שלם, יש לעגלו לשלם הקרוב ביותר.**
- **במידה והסכום קטן מ-0 יש להתייחס אליו כאל 0, ואם הוא גדול מ-255 יש להתייחס אליו כאל 255.**
- **בחישוב ערך לפיקסל x הנמצא על גבולות התמונה, יש להתייחס לערכי פיקסלים הנמצאים מחוץ לגבולות תמונת המקור כאילו היו בעלי ערך זהה לזה של הפיקסל x.**

Resize

במקרים רבים נרצה להיות מסוגלים לשנות את גודל התמונה. כפי שראינו בסרטון, בביצוע resize אנחנו בודקים עבור כל פיקסל בתמונת היעד (בגודל החדש) מה הנקודה שבה הוא "נופל" בתמונת המקור, וערכו החדש מחושב כצירוף של 4 הפיקסלים הקרובים לנקודה זו. הערך מחושב באמצעות אינטרפולציה, לפי החישוב הבא:



$$a \cdot (1-\Delta x)\cdot(1-\Delta y) + b \cdot \Delta y \cdot (1-\Delta x) + c \cdot \Delta x \cdot (1-\Delta y) + d \cdot \Delta x \cdot \Delta y$$

כאשר a, b, c, d הם ערכי הפיקסלים בתמונת המקור הקרובים לנקודה שבה הפיקסל מתמונת היעד "נופל".

לדוגמא:

נתבונן בתמונה הבאה, שגודלה 5 x 3 פיקסלים, ונרצה לבצע לה resize לגודל 7 x 6 פיקסלים.

15	30	45	60	75
90	105	120	135	150
165	180	195	210	225

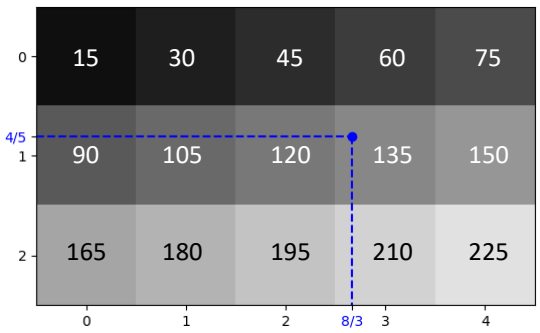
ראשית עלינו להבין מה המקור של כל פיקסל בתמונת היעד. לצורך כך עלינו לחשב איפה הוא נמצא ביחס לאורך ולרוחב, ולהשתמש במיקום היחסי כדי להבין איפה הוא נופל בתמונת המקור.

ניקח לדוגמא את הפיקסל `[y, x] = [2, 4]` בתמונת היעד. ערכי ה-y של תמונת היעד הם בטווח [0,5] ולכן מיקומו היחסי ביחס לציר ה-y הוא $^{2}/_{5}$. באופן דומה מיקומו היחסי ביחס לציר ה-x הוא $^{4}/_{6}$.

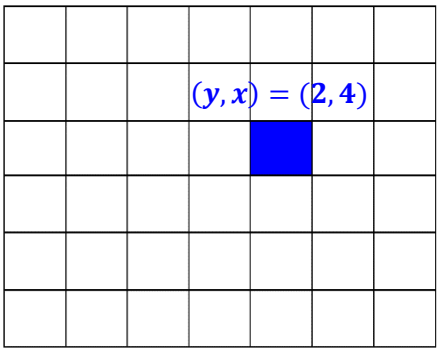
כעת נותר לחשב מה הקואורדינטה בתמונת המקור שמתאימה למיקום היחסי שמצאנו. לצורך כך נכפול את המיקום היחסי של y ב-2 (מפני שערכי ה-y של תמונת המקור הם בטווח [0,2]) ואת המיקום היחסי של x ב-4.

בסה"כ נקבל שהפיקסל נופל בנקודה `[y, x] = [4/5, 8/3]` בתמונת המקור.

Source Image



Destination Image



כעת נמצא את הערכים הדרושים עבור החישוב:

$$\Delta x = \frac{2}{3} \qquad \Delta y = \frac{4}{5} \qquad \underbrace{a = 45}_{image[0,2]} \qquad \underbrace{b = 120}_{image[1,2]} \qquad \underbrace{c = 60}_{image[0,3]} \qquad \underbrace{d = 135}_{image[1,3]}$$

נציב בנוסחא ונקבל:

$$new_image[2,4] = 45\left(1-\frac{2}{3}\right)\left(1-\frac{4}{5}\right) + 120\cdot\frac{4}{5}\cdot\left(1-\frac{2}{3}\right) + 60\cdot\frac{\mathbf{2}}{3}\cdot\left(1-\frac{4}{5}\right) + 135\cdot\frac{\mathbf{2}}{3}\cdot\frac{4}{5} = 115$$

כלומר ערכו של הפיקסל `[y, x] = [2, 4]` בתמונת היעד יהיה 115.

שימו לב שהערכים של Δx ושל Δy הם בין 0 ל-1 **<u>כולל</u>**.

5. ממשו את הפונקציה:

`bilinear_interpolation(image, y, x)`

המקבלת תמונה בעלת ערוץ צבע יחיד (רשימה דו-מימדית) ואת הקואורדינטות של פיקסל מתמונת היעד כפי שהן "נופלות" בתמונת המקור (y הקואורדינטה לאורך התמונה, כלומר בשורות ו-x לרוחב התמונה, כלומר בעמודות) ומחזירה את ערך אותו הפיקסל (מספר שלם בין 0 ל-255) לפי חישוב האינטרפולציה שהצגנו לעיל.

ניתן להניח שהפונקציה מקבלת תמונה חוקית בעלת ערוץ צבע יחיד (רשימה דו-מימדית).

ניתן להניח שהקואורדינטות x,y הן בתוך גבולות התמונה (ויכולות גם להיות על הגבולות ממש).

לדוגמא:

```
bilinear_interpolation([[0, 64], [128, 255]], 0, 0) → 0
bilinear_interpolation([[0, 64], [128, 255]], 1, 1) → 255
bilinear_interpolation([[0, 64], [128, 255]], 0.5, 0.5) → 112
bilinear_interpolation([[0, 64], [128, 255]], 0.5, 1) → 160
bilinear_interpolation([[15, 30, 45, 60, 75], [90, 105, 120, 135, 150],
[165, 180, 195, 210, 225]], 4/5, 8/3) → 115
```

<u>שימו לב</u>:

- **הקואורדינטות x,y יכולות להיות מורכבות ממספרים לא שלמים.**
- **במידה והערך המתקבל מהחישוב אינו שלם, יש לעגלו לשלם הקרוב ביותר.**

6. ממשו את הפונקציה:

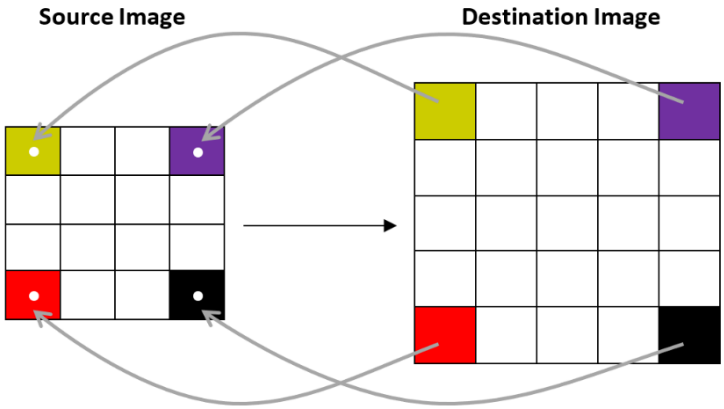
`resize(image , new_height, new_width)`

המקבלת תמונה בעלת ערוץ צבע יחיד (רשימה דו-מימדית) ושני מספרים שלמים, ומחזירה תמונה חדשה בגודל new_width × new_height כך שערכו של כל פיקסל בתמונה המוחזרת מחושב בהתאם למיקומו היחסי בתמונת המקור.

מיקומו במקור יחושב באופן הבא:

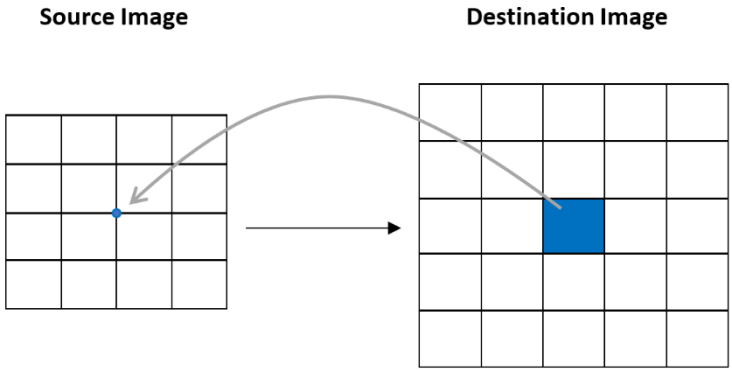
1. פינות ימופו לפינות.

לדוגמא הפיקסל בפינה השמאלית העליונה (בשורה האפס ובעמודה האפס) בתמונת היעד, ימופה לפיקסל בפינה השמאלית העליונה (בשורה האפס ובעמודה האפס) בתמונת המקור: $(0,0) \rightarrow (0,0)$.



2. כל נקודה אחרת (שאינה פינה), תמופה באופן פרופורציונלי למיקומה ביחס לפינות.

לדוגמא עבור תמונת מקור מגודל 4×4, הפיקסל האמצעי בתמונת יעד מגודל 5×5 (זה שבשורה 2 ועמודה 2) ימופה להיות בדיוק במרכז של תמונת המקור: $(1.5,1.5) \rightarrow (2,2)$.



ניתן להניח שהפונקציה מקבלת תמונה דו-מימדית חוקית (ערוץ צבע יחיד) וכי המימדים החדשים הינם מספרים שלמים וחיוביים הגדולים מ-1.

אין לשנות את תמונת המקור.

סיבוב ב-90 מעלות

7. ממשו את הפונקציה:

`rotate_90(image, direction)`

המקבלת תמונה (צבעונית או בעלת ערוץ יחיד) וכיוון (מחרוזת שהיא 'R' או 'L') ומחזירה תמונה דומה, מסובבת ב-90 מעלות לכיוון המבוקש.

ניתן להניח שהפונקציה מקבלת תמונה חוקית ושהקלט direction תקין. שימו לב שהתמונה יכולה להיות צבעונית – תלת מימדית, או בעלת ערוץ יחיד – דו-מימדית, על הקוד שלכם לתמוך בשני המקרים.

אין לשנות את תמונת המקור.

לדוגמא:

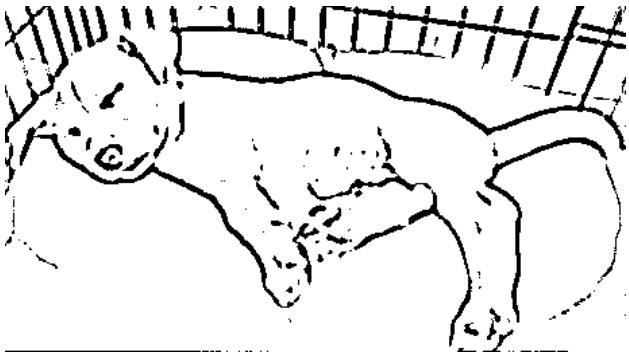
```
rotate_90([[1, 2, 3],          [[4, 1],
           [4, 5, 6]], 'R')  →   [5, 2],
                                 [6, 3]]

rotate_90([[1, 2, 3],          [[3, 6],
           [4, 5, 6]], 'L')  →   [2, 5],
                                 [1, 4]]

rotate_90([[[1, 2, 3], [4, 5, 6]],          [[[4, 5, 6], [255, 200, 7]],
           [[0, 5, 9], [255, 200, 7]]], 'L')  →  [[1, 2, 3], [0, 5, 9]]]
```

זיהוי גבולות





גבול בתמונה הוא קו מתאר של אובייקט מסויים בה. בהרבה בעיות שונות בעיבוד תמונה נרצה לדעת לזהות ולבודד את קווי המתאר הללו, וישנן מספר שיטות לזיהוי גבולות. שיטות אלה מתבססות על מעברים חדים בין גוונים. מעבר חד בגוונים מעיד על שינוי בתמונה, כלומר הופעה של אובייקט חדש, ועל כן הם מצביעים על גבול. ראינו בסרטון שיטה לזיהוי גבולות הנקראת adaptive threshold אותה נממש כעת. נזכיר בקצרה, ב-adaptive threshold אנו מחשבים לכל פיקסל ערך סף שהוא ממוצע השכנים שלו בסביבה מסויימת (בגודל block_size × block_size) פחות קבוע כלשהו c. אם פיקסל מסויים כהה מהסביבה שלו, סימן שסביבו יש פיקסלים שערכם גבוה משלו (שכן שחור זה 0 ולבן זה 255) ולכן ערכו יהיה קטן מהממוצע בסביבתו. הקבוע c אותו אנו מפחיתים מהערך הממוצע מאפשר לנו לדרוש שהפיקסל יהיה כהה משמעותית מהסביבה שכן ערכו צריך להיות אפילו קטן מהממוצע פחות אותו קבוע.

8. ממשו את הפונקציה:

`get_edges(image, blur_size, block_size, c)`

המקבלת תמונה בגווני שחור לבן (רשימה דו-מימדית) ושלושה מספרים, ומחזירה תמונה חדשה, בעלת אותם מימדים, המורכבת <u>משני ערכים בלבד</u> (שחור ולבן) כאשר פיקסלים שחורים מסמנים גבולות בתמונה.

ניתן להניח שהפונקציה מקבלת תמונה דו-מימדית חוקית (ערוץ צבע יחיד), ש-block_size ו-blur_size הם מספרים שלמים, חיוביים **<u>ואי-זוגיים</u>** וש-c הוא מספר אי-שלילי.

<u>אין לשנות את תמונת המקור.</u>

לדוגמא:

```
get_edges([[200, 50, 200]], 3, 3, 10) → [[255, 0, 255]]
```

<u>שימו לב</u>:

- **בשביל למצוא גבולות בתמונה עליכם לטשטש תחילה את התמונה באמצעות הפרמטר `blur_size`.**
- **נסמן $r = block_size \;//\; 2$, אז הנוסחא ל-$threshold$ היא:**

$$threshold[i][j] = avg(blurred_image[i-r:i+r+1][j-r:j+r+1]) - c$$

שימו לב, זהו ביטוי מופשט – לא קוד בפייתון, הביטוי מתאר ממוצע של ערכי התאים הנמצאים בריבוע עם צלע מאורך block_size תאים סביב התא ה i,j בתמונה, פחות הערך c.

(בחישוב הממוצע, יש להתייחס לערכי פיקסלים הנמצאים מחוץ לגבולות התמונה כאילו היו בעלי ערך זהה לזה של הפיקסל אותו אתם בודקים בתמונה המטושטשת).

- **אם $blurred_image[i][j] < threshold[i][j]$ אז $new_image[i][j]$ יהיה שחור ואחרת לבן.**
- **כדי להימנע מכפל קוד, היעזרו בפונקציות קודמות שמימשתם ככל הניתן.**

צמצום מספר הצבעים בתמונה (קוונטיזציה)



בתהליך הקוונטיזציה (Quantization), אנחנו מצמצמים מגוון של ערכים לכדי ערך בודד – למשל בוחרים גוון ספציפי של אדום שיחליף 10 גוונים שונים. למעשה אנחנו מחלקים את 256 הגוונים שלנו למספר מסויים של גוונים.

קיימים מספר אלגוריתמים מתחום למידת המכונה שמטרתם לבחור את הגוונים האופטימליים שנרצה לשמור, ואת הדרך הנכונה לשייך כל גוון מקורי לגוון החדש, את אלגוריתמים אלה עוד תראו בקורסים עתידיים. אנחנו נשתמש בחישוב פשוט יותר אשר בוחר N גוונים במרחקים שווים, מבלי לבחור גוונים אופטימליים, ומעביר כל גוון בתמונה המקורית לגוון הכי קרוב אליו מבין N הגוונים שבחרנו להשאיר. נשתמש בנוסחא:

$$qimg[i][j] = round\left(floor\left(img[i][j] \cdot \frac{N}{256}\right) \cdot \frac{255}{N-1}\right)$$

9. ממשו את הפונקציה:

`quantize(image, N)`

שמקבלת תמונה כרשימה דו-מימדית (בעלת ערוץ צבע יחיד) ומחזירה תמונה בעלת מימדים זהים, בה הערכים של הפיקסלים מחושבים לפי הנוסחא לעיל.

ניתן להניח שהפונקציה מקבלת תמונה דו-מימדית חוקית (ערוץ צבע יחיד) וכי N הוא מספר טבעי <u>גדול מ-1</u>.

<u>אין לשנות את תמונת המקור.</u>

לדוגמא:

```
quantize([[0, 50, 100], [150, 200, 250]], 8) → [[0, 36, 109], [146, 219, 255]]
```

לקוונטיזציה של תמונה צבעונית יש להפעיל את הקוונטיזציה שמימשנו עבור תמונה בעלת צבע יחיד על כל ערוץ צבע בנפרד, לכן בתמונה הסופית מספר הגוונים יהיה $N^{\#channels}$ (וודאו שאתם מבינים מדוע).

10. ממשו את הפונקציה:

`quantize_colored_image(image, N)`

שמקבלת תמונה צבעונית (רשימה תלת-מימדית) ומחזירה תמונה דומה לאחר קוונטיזציה ל-$N^{\#channels}$ גוונים.

חישבו – באלו מהפונקציות שכתבתם עד כה עליכם להשתמש?

ניתן להניח שהפונקציה מקבלת תמונה צבעונית חוקית, וכי N הוא מספר טבעי גדול מ-1.

אין לשנות את תמונת המקור.

תכנית לעריכת תמונות

בחלק זה עליכם לכתוב את התכנית הכללית שמאפשרת לערוך תמונות, תוך שימוש בפונקציות שמימשתם עד כה.

חלק זה ייכתב תחת:

`if __name__=="__main__":`

הרצת התכנית תתבצע ע"י הפקודה:

`python3 image_editor.py <image_path>`

כאשר הארגומנט **`image_path`** שמתקבל משורת הפקודה מייצג נתיב של תמונה אותה התכנית תערוך.

יש לוודא שמספר הארגומנטים תקין.

במידה ומספר הארגומנטים שונה מהמצופה - יש להדפיס הודעת שגיאה ולסיים את הריצה.

ניתן להניח שהנתיב שהתקבל תקין ומכיל תמונה חוקית.

על התכנית לעבוד באופן הבא:

א. טעינת התמונה

בשלב הראשון נטען את התמונה שהניתוב שלה הוא **`image_path`**, באמצעות הפונקציה **`load_image`** מקובץ העזר.

זכרו שפונקציה זו מחזירה ייצוג של התמונה ברשימה תלת-מימדית (RGB).

ב. עריכת התמונה

בשלב הבא תוצג למשתמש הודעה לבחירתכם שבה יופיע פירוט של הפעולות אותן ניתן לבצע על התמונה, והמשתמש יכניס את מספר הפעולה אותה הוא רוצה לבצע.

אם הקלט תקין (כלומר מספר הפעולה מתאים לאחת הפעולות שמפורטות בהמשך)- נבצע את הפעולה על התמונה שנבחרה, ונמשיך לעבוד עם התמונה המעודכנת.

אם הקלט לא תקין- תודפס הודעה לבחירתכם והמשתמש יתבקש להכניס שוב את מספר הפעולה הרצויה.

לאחר מכן נחזור שוב על התהליך (בלומר, נציג למשתמש שוב את פירוט הפעולות ונאפשר לו לבצע פעולה על התמונה באותו אופן), עד שהוא יבחר לצאת מהתכנית.

הפעולות אותן ניתן לבצע הן:

1. **המרת תמונה צבעונית לתמונת גווני-אפור**

במידה והתמונה כבר בגווני אפור- תודפס הודעה לבחירתכם והתמונה תישאר כמו שהיא.

2. **טשטוש התמונה**

לאחר בחירה באפשרות זו תוצג למשתמש הודעה בה הוא יתבקש להכניס את גודל הקרנל שאיתו נבצע את הטשטוש.

יש לוודא שהערך שהתקבל הוא מספר שלם, חיובי ואי-זוגי. במידה ולא- יש להדפיס הודעה לבחירתכם ולא לבצע את הפעולה.

על מנת לטשטש תמונה צבעונית- נפריד אותה לערוצי צבע, נטשטש כל ערוץ צבע בנפרד ולבסוף נחבר אותם בחזרה.

3. **שינוי גודל התמונה**

לאחר בחירה באפשרות זו תוצג למשתמש הודעה בה הוא יתבקש להכניס שני ערכים מופרדים בפסיק (","), כאשר הערך הראשון הוא הגובה החדש של התמונה, והערך השני הוא הרוחב החדש שלה.

יש לוודא שהתקבלו שני מספרים שלמים וחיוביים הגדולים מ-1, בפורמט תקין (כלומר מופרדים בפסיק).

במידה והפורמט לא תקין או שהערכים לא תקינים- יש להדפיס הודעה לבחירתכם ולא לבצע את הפעולה.

4. **סיבוב התמונה ב- 90 מעלות**

לאחר בחירה באפשרות זו תוצג למשתמש הודעה בה הוא יתבקש להכניס את כיוון הסיבוב ('R' או 'L').

יש לוודא שהערך שהתקבל הוא 'R' או 'L'. במידה והערך שונה- יש להדפיס הודעה לבחירתכם ולא לבצע את הפעולה.

5. **יצירת תמונת קווי המתאר (edges) של התמונה המקורית**

לאחר בחירה באפשרות זו תוצג למשתמש הודעה בה הוא יתבקש להכניס 3 ערכים מופרדים בפסיק (","), כאשר הערך הראשון הוא blur_size, השני block_size והשלישי c.

יש לוודא שהערכים התקבלו בפורמט תקין (כלומר 3 ערכים מופרדים בפסיק), וכמו כן יש לוודא ששני הערכים הראשונים שהתקבלו הם מספרים שלמים, חיוביים ואי-זוגיים ושהערך השלישי הוא מספר אי-שלילי.

במידה והפורמט לא תקין או שהערכים לא תקינים- יש להדפיס הודעה לבחירתכם ולא לבצע את הפעולה.

במידה והתמונה צבעונית- נמיר אותה קודם לתמונת גווני-אפור ולאחר מכן ניצור את תמונת קווי המתאר.

6. **ביצוע קוונטיזציה לתמונה**

לאחר בחירה באפשרות זו תוצג למשתמש הודעה בה הוא יתבקש להכניס את מספר הגוונים הרצוי לאחר הקוונטיזציה.

אם התמונה צבעונית- המספר שיוכנס יתייחס לכל ערוץ צבע בפני עצמו.

לדוגמא: אם התמונה צבעונית והמשתמש ביקש לבצע קוונטיזציה ל-5 גוונים- עליכם להפריד את התמונה לערוצי הצבע השונים, לבצע **לכל ערוץ** קוונטיזציה ל-5 גוונים, ולחבר את הערוצים לאחר הקוונטיזציה.

יש לוודא שהערך שהתקבל מהמשתמש הוא מספר שלם וחיובי הגדול מ-1. במידה ולא - יש להדפיס הודעה לבחירתכם ולא לבצע את הפעולה.

7. **הצגת התמונה**

באמצעות שימוש בפונקציה **`show_image`** מקובץ העזר.

8. **יציאה מהתכנית**

אם נבחרה פעולה שדורשת קבלת קלט נוסף מהמשתמש- עליכם לוודא תקינות הקלט (כלומר לוודא שהערכים תקינים ושהם התקבלו בפורמט תקין). במידה והקלט לא תקין- יש להדפיס הודעת שגיאה לבחירתכם ולא לבצע את הפעולה (במקרה זה נציג שוב למשתמש את פירוט הפעולות ונאפשר לו להמשיך לערוך את התמונה).

שימו לב שמלבד הפעולה הראשונה, יתר הפעולות צריכות לתמוך גם בתמונות צבעוניות וגם בתמונות עם ערוץ צבע יחיד.

ג. יציאה מהתכנית

לאחר שהמשתמש בוחר לצאת מהתכנית, תוצג לו הודעה שבה הוא יתבקש להכניס נתיב שבו התמונה תישמר. עליכם לשמור את התמונה בנתיב שהתקבל ע"י שימוש ב- **save_image** מקובץ העזר, ולסיים את התכנית.

<u>שימו לב:</u>

- **מומלץ מאוד לקרוא את ההוראות בצורה יסודית עד שאתם מבינים בדיוק איך התכנית צריכה לעבוד.**
- **יש הרבה תרחישים אפשריים (בהתאם לקלט שהמשתמש מכניס). חישבו על כמה שיותר תרחישים, וודאו שאתם מתמודדים עם כולם.**
- **לאחר כל פעולת עריכה זכרו להמשיך לעבוד עם התמונה המעודכנת.**
- **מומלץ מאוד לפצל לפונקציות ולעבוד כמה שיותר מסודר. השתדלו להימנע ככל הניתן מכפל קוד על ידי שימוש בפונקציות שכבר מימשתם.**

הוראות הגשה

עליכם להגיש קובץ בשם **ex6.zip** בקישור ההגשה של תרגיל 6 דרך אתר הקורס על ידי לחיצה על "Upload file". הקובץ **ex6.zip** צריך להכיל אך ורק את הקובץ **image_editor.py**.

לפני שאתם מתחילים – טיפים והנחיות

- הקפידו לכתוב תיעוד לקוד שלכם ובפרט לכל פונקציה שאתם כותבים.
- לרשותכם התיקייה examples, המכילה דוגמאות של תמונות איתן אתם יכולים לבדוק את התכנית שלכם.
- אנו מעודדים אתכם לבחון את התרגיל גם עם תמונות שלכם! מכיוון שזמן הריצה תלוי במספר הפיקסלים אנו ממליצים לעבוד בתחילת התרגיל עם תמונות קטנות, או להקטין את התמונות באמצעות הפונקציה resize.
- בתרגיל זה, כל פעולה על תמונה צבעונית תבוצע על כל ערוץ צבע בנפרד (אלא אם צוין אחרת). באופן זה ניתן לנצל פונקציות שתומכת בערוץ צבע יחיד גם בעבור תמונות צבעוניות.
- ניתן להניח תקינות הקלטים לכל אחד מהסעיפים (בהתאם להגדרות הפרטניות של כל סעיף). בפרט, ניתן להניח כי כל התמונות ניתנות בפורמט תקין (רשימה של רשימות, שבכל אחת מהן אותו מספר פיקסלים), וכי כל הרשימות הן אכן רשימות לא ריקות.
- בכל הפונקציות בתרגיל זה <u>אין לבצע שום שינוי</u> בתמונות הקלט, אלא להחזיר תמונות חדשות!
- לצורך פיתרון התרגיל תוכלו להשתמש במודולים sys , copy ו-math. **אין להשתמש** במודולים numpy או PIL.

בהצלחה!